

Comunicação entre processos

Samuel Francisco Ferrigo

Introdução

A comunicação entre processos é importante pois evita que ocorram diversos problemas, tais como:

- a entrada de um processo B depende da saída de um processo A
- condições de corrida, como o acesso simultâneo à memória compartilhada

A parte de um programa que está tendo acesso a uma memória ou arquivo compartilhado é chamada de **região crítica**

Condições para evitar a região crítica

Todas as condições abaixo devem estar satisfeitas:

- Dois processos nunca podem estar simultaneamente em suas regiões críticas
- Nada pode ser afirmado sobre a velocidade ou sobre o número de processadores
- Nenhum processo deve esperar eternamente para entrar em sua região crítica
- Nenhum processo executando fora de sua região crítica pode bloquear outros processos

Conceitos relacionados

Determinismo

Abordagens de comunicação

- Via kernel
- Via memória compartilhada

Formas de controle à região crítica

- Espera ociosa (ou ativa)
- Dormir e acordar (uso de chamadas de sistema)

Determinismo

Um algoritmo determinístico é aquele que sempre produz o mesmo resultado/saída

Porém, quando trabalhamos com processos e, conseqüentemente, com concorrência, deparamos com situações onde o resultado depende do contexto onde está sendo executado

No caso de processos, esse determinismo está diretamente associado ao tempo, pois este é compartilhado

Outras situações onde o contexto interfere no resultado são

- Leitura de um valor do usuário
- Variáveis globais
- Erro de hardware

Abordagens de comunicação

Comunicação via memória compartilhada

Tipo de comunicação onde dois ou mais processos compartilham uma mesma região de memória

Podem ler e escrever nessa região para trocar informações entre si

Pode ser usada para transmitir grandes quantidades de dados rapidamente

Requer sincronização para evitar conflitos de acesso

Essa sincronização requer algum tipo de comunicação

Comunicação via kernel

Há várias formas de comunicação entre processos utilizando o kernel, como

- Pipes
- Sinais
- Troca de Mensagens
- Remote Procedure Call

Condições para concorrência

Dois processos não podem executar dentro da região crítica ao mesmo tempo

Não pode haver nenhuma suposição sobre a velocidade de execução ou número de processadores

Nenhum processo fora da região crítica pode bloquear outro processo

Não pode haver espera “infinita” para entrar na região crítica

Formas de controle à região crítica

Espera ociosa

É uma forma de espera do processo onde ele fica em execução não fazendo nada de útil

Consome tempo de CPU

Fácil de Implementar

Dormir e acordar

Neste método, caso um processo tenha que aguardar na região crítica, este é colocado para dormir, ou seja, é bloqueado

Este processo, então somente é acordado quando a região crítica volta a ficar disponível

Esta técnica consome muito menos recursos

É a técnica normalmente utilizada

Formas de comunicação

Formas de comunicação

Pipes

Sinais

Fila de mensagens

Semáforos

Mutex

Barreiras

Sockets

Mutex

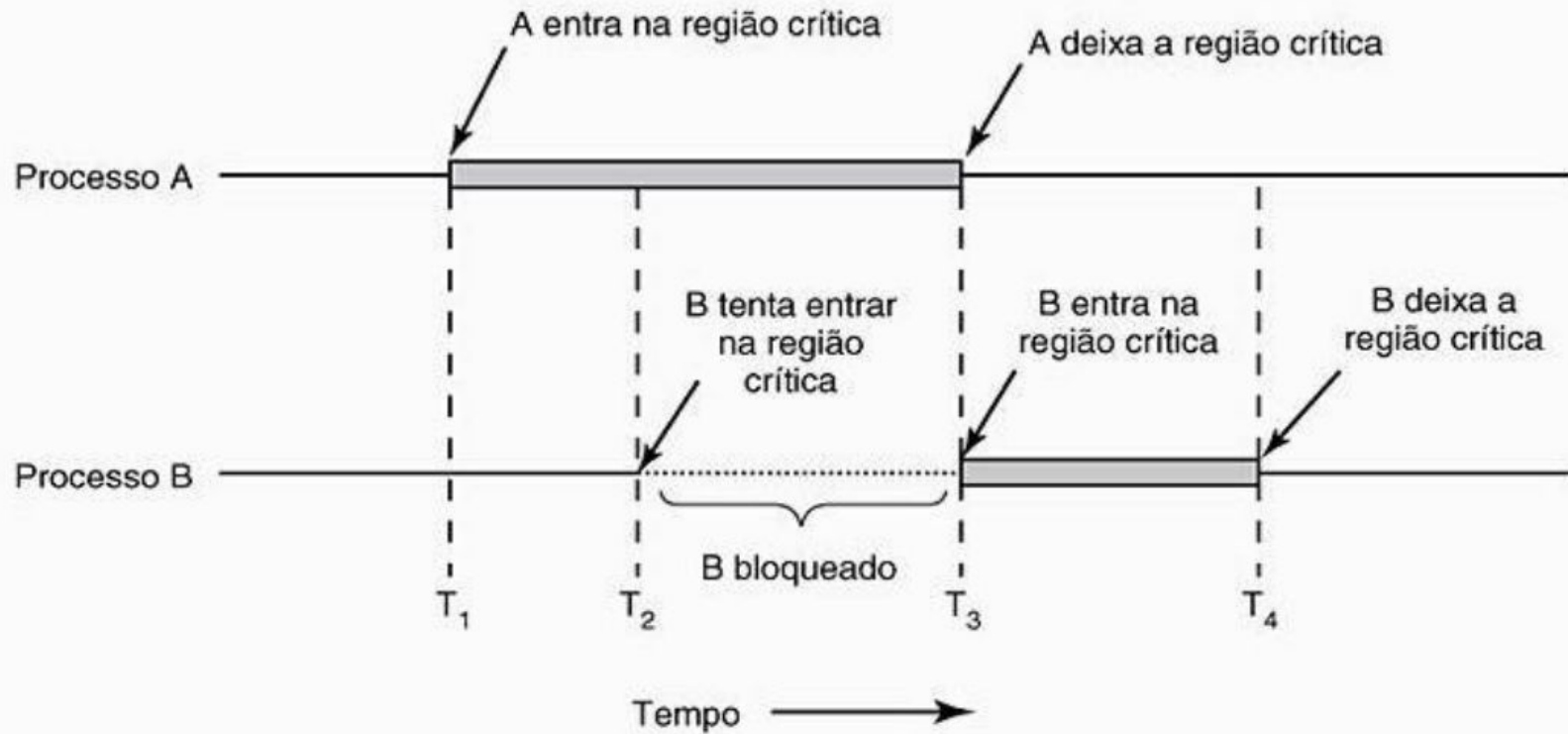
A exclusão mútua (Mutex) é um tipo de semáforo onde apenas um processo por vez acessa a memória compartilhada

Um exemplo de exclusão mútua pode ser aplicada com o algoritmo de Peterson

- Este tem uma espera ociosa

Os processadores contam com uma instrução que faz esse controle à região compartilhada

- Uma delas é a instrução TSL (Test-and-Set-Lock), onde a CPU impede o acesso ao barramento da memória quando há um processo em execução no processador
- Trata-se de uma instrução atômica, pois é executada inteiramente sem interrupção e não pode ser dividida em operações menores



Lógica da exclusão mútua. Fonte. TANEMBAUM, A. Sistemas operacionais Modernos, 3ª Ed., 2009

Mutex em Pthreads

Incluso na biblioteca <pthread.h>

Primeiro cria-se o lock

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

- lock: variável que contém a inicialização do MUTEX (PTHREAD_MUTEX_INITIALIZER)

Mutex em Pthreads

Para identificação da seção crítica utiliza-se, para demarcar o início:

```
pthread_mutex_lock(&lock);
```

E, para identificar seu final:

```
pthread_mutex_unlock(&lock);
```

Pipes

Comunicação unidirecional entre dois processos

A saída de um processo é a entrada do outro

Usados para transmitir pequenas quantidades de dados entre processos, como mensagens ou comandos

O bash permite o pipe entre processos, através do caractere |

Sinais

É uma forma de interrupção de um processo pelo sistema operacional

- Ctrl+C - SIGINT - Interrupção no Linux
- Ctrl+Z - SIGTSTP - Parada de um processo via terminal no Linux
- SIGKILL - Sinal enviado pelo kill do Linux

Podem ser usados para enviar mensagens entre processos

O processo receptor pode tratar o sinal como uma notificação para executar uma ação específica

Fila de mensagens

Permitem que processos enviem e recebam mensagens de tamanho variável de forma assíncrona

As mensagens são armazenadas em uma fila gerenciada pelo sistema operacional

O processo receptor pode ler as mensagens da fila quando estiver pronto

Semáforos

Uma das formas de sincronização entre processos

Usado para controlar o acesso a recursos compartilhados

Permite que um processo bloqueie outro até que um recurso compartilhado esteja disponível

Em comparação com uma fila de mensagens, um semáforo é usado para sincronização de acesso a recursos compartilhados em um sistema

Já a fila de mensagens é usada para comunicação assíncrona entre processos

Semáforos em C

Utiliza a biblioteca <semaphore.h>

O semáforo é criado no código com

```
sem_t semaforo;
```

- **semáforo:** variável que controlará o semáforo. Para dois ou mais recursos a serem controlados, deve-se utilizar um vetor de tamanho N, onde N é a quantidade de recursos a serem controlados pelo semáforo.

Semáforos em C

Em seguida, o semáforo deve ser inicializado dentro da função `main()`{

```
sem_init(&semaforo, compartilhado, valor);
```

- `&semaforo` é o endereço de memória que contém o semáforo
- `compartilhado` indica se será compartilhado entre threads (valor 0) ou entre processos (valor 1)
- `valor` indica a quantidade de threads/processos que podem acessar simultaneamente o recurso

Semáforos em C

Para controle da seção crítica deve-se usar, para alocar o recurso:

```
sem_wait(&semaforo);
```

E, para liberar:

```
sem_post(&semaforo);
```

Ao final, o semáforo deve ser destruído utilizando

```
sem_destroy(&semaforo);
```

Barreiras

A barreira é um objeto que bloqueia a execução até que um determinado número de processos chegue a ela

Quando o número determinado é atingido, a barreira é liberada e todos os processos podem continuar a execução

Permite que um grupo de processos ou threads aguarde uns pelos outros em um ponto específico do código antes de continuar a execução

Barreiras em pthreads

Incluso na biblioteca <pthread.h>

Primeiro cria-se a barreira

```
pthread_barrier_t barreira;
```

- `barreira`: **variável** que contém a barreira;

Barreiras em pthreads

Inicia-se a barreira com

```
pthread_barrier_init(&barreira, attr, n_threads);
```

- `&barreira` é o endereço da barreira criada;
- `attr` são os atributos da barreira;
- `n_threads` é o número de threads a serem sincronizadas;

Barreiras em pthreads

Depois identifica(m)-se o(s) trecho(s) onde se deseja(m) aplicar a barreira com

```
pthread_barrier_wait(&barreira);
```

- `&barreira` é o endereço da barreira criada;

Por fim, a barreira é finalizada com

```
pthread_barrier_destroy(&barreira);
```

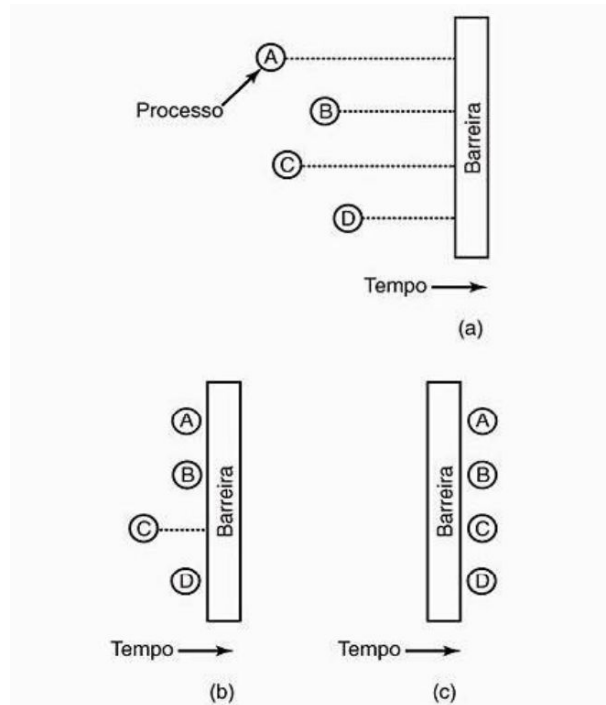


Figura 2.30 Uso de uma barreira. (a) Processos se aproximando de uma barreira. (b) Todos os processos, exceto um, estão bloqueados pela barreira. (c) Quando o último processo chega à barreira, todos passam por ela.

Funcionamento das barreiras. Fonte. TANENBAUM, A. Sistemas operacionais Modernos, 3ª Ed., 2009

Sockets

Forma de comunicação entre processos que utilizam redes de computadores

Realiza troca de mensagens através de diretivas `send()` e `receive()`

Os sockets geralmente são TCP ou UDP e fazem uso do protocolo IP para comunicação entre dispositivos.